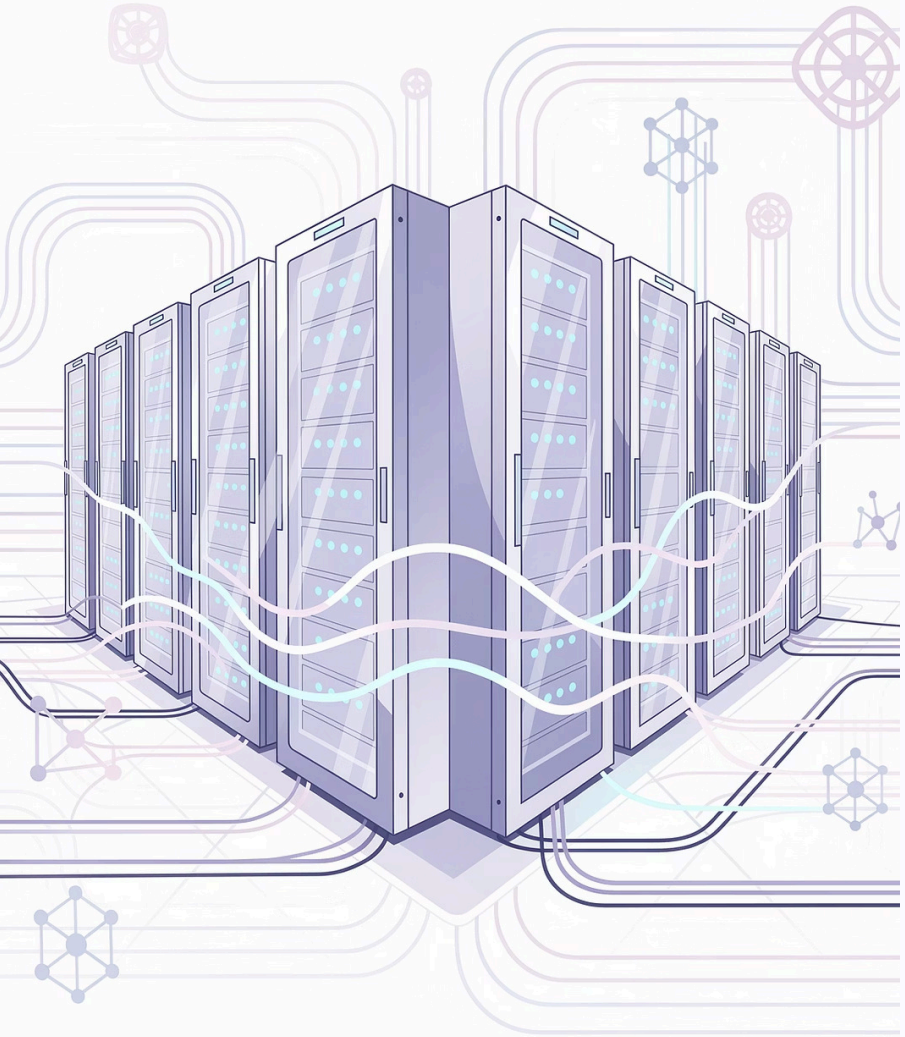
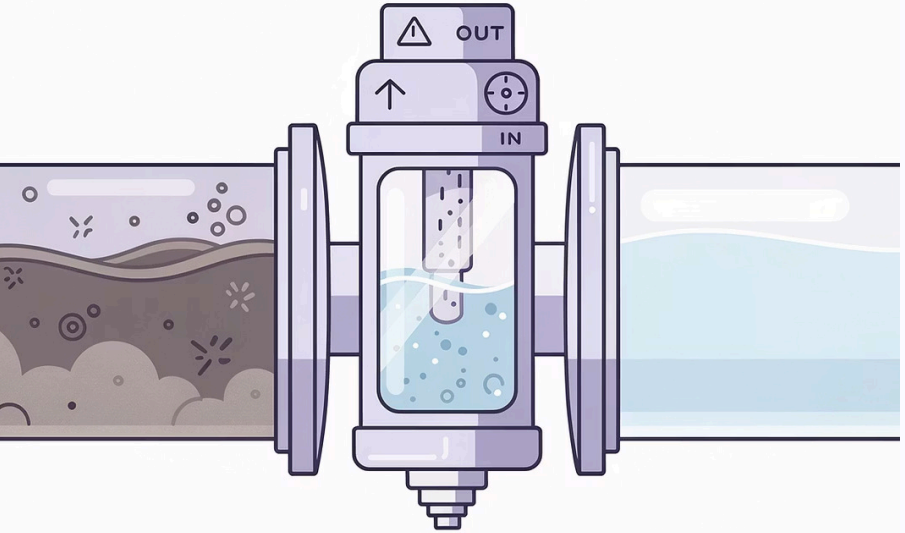


Data Sources & Data Management

A practical deep-dive into how data enters your pipelines, how it is stored, and how sound management principles keep your architecture reliable, scalable, and production-ready.

DATA ENGINEERING APACHE SPARK



Why This Matters

The Golden Rule of Data Engineering

Every downstream insight, model, and decision is only as trustworthy as the data that feeds it. Flawed data flowing silently through a pipeline is far more dangerous than a pipeline that fails loudly.

Where Quality is Won or Lost

Data quality is not a problem you can fix at the reporting layer. It must be enforced at the point of ingestion — before raw records ever touch your transformation logic.

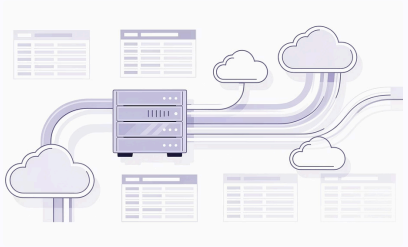
- **Garbage in → garbage out** — corrupted inputs produce misleading outputs
- Schema mismatches silently drop or coerce records
- Late or duplicate events distort aggregations

Types of Data Sources

Modern data platforms draw from a wide variety of source systems. Understanding the characteristics of each helps you choose the right ingestion strategy.

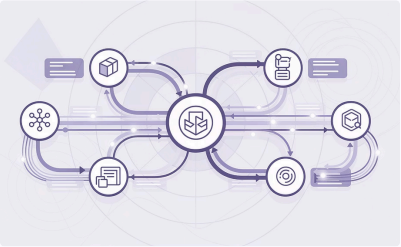
Files

Flat and semi-structured files remain one of the most common ingestion formats. **CSV** for raw tabular exports, **JSON** for nested API responses, and **Parquet** for columnar analytics workloads.



Databases

Relational and cloud-native databases such as **Amazon RDS** and **Snowflake** provide structured, queryable data with strong consistency guarantees — ideal for JDBC-based batch ingestion.



Streams

Apache Kafka and similar event-streaming platforms deliver continuous, low-latency data flows. Essential for real-time analytics, fraud detection, and event-driven architectures.

Batch vs. Streaming Sources

Choosing between batch and streaming ingestion is one of the most consequential architectural decisions in a data platform. Each model has distinct trade-offs across latency, complexity, and cost.

⏱️ Batch Processing

- Data collected and processed at scheduled intervals (hourly, daily)
- Higher latency — insights are retrospective, not real-time
- Lower infrastructure complexity and cost
- Ideal for reporting, ETL pipelines, and historical analysis
- Sources: JDBC databases, file drops, scheduled API exports

⚡ Streaming Processing

- Data processed continuously as events arrive
- Sub-second to second-level latency — near real-time insights
- Higher operational complexity; requires stateful processing
- Ideal for fraud detection, live dashboards, IoT telemetry
- Sources: Kafka topics, Kinesis streams, CDC event logs

📌 Streaming pipelines with Structured Streaming will be explored in an upcoming deep-dive session.

Reading Data — DataFrameReader

The Unified Read API

Spark's DataFrameReader is the single entry point for loading data from virtually any source into a distributed DataFrame. The fluent API keeps ingestion code consistent regardless of source type.

```
spark.read
  .format("parquet")
  .option("header", "true")
  .option("inferSchema", "true")
  .load("s3://bucket/path/")
```

Key Options

- .format() — specifies the connector: csv, json, parquet, delta, jdbc
- .option() — passes source-specific configuration (credentials, schema hints, delimiters)
- .schema() — enforces an explicit schema instead of inferring it at runtime
- .load() — triggers the read from the specified path or table

> Always prefer explicit schemas over inferSchema in production — inference requires an extra full scan of the data.

Writing Data — DataFrameWriter

The Unified Write API

The `DataFrameWriter` mirrors the read API and gives you fine-grained control over how, where, and in what mode your transformed data is persisted. Choosing the correct write mode is critical to pipeline correctness.

```
df.write
  .format("delta")
  .mode("overwrite")
  .partitionBy("year", "month")
  .save("s3://bucket/output/")
```

Write Modes Explained

overwrite

Replaces all existing data at the target path. Use for full-refresh loads.

append

Adds new records without modifying existing data. Use for incremental loads.

errorIfExists

Default mode — fails if data already exists at the target location.

ignore

Silently skips the write if the target already exists.

Choosing the Right File Format

File format selection has a direct impact on query performance, storage cost, and pipeline compatibility. Use this guide to match the right format to the right use case.

Format	Primary Use Case	Key Characteristics	Best For
CSV	Raw ingestion	Human-readable, no compression, no schema enforcement	Landing zone, external system exports
JSON	Nested / semi-structured data	Schema-flexible, supports arrays and objects, verbose	API responses, event logs
Parquet	Analytical queries	Columnar storage, highly compressed, predicate pushdown	Data warehouses, large-scale aggregations
Delta	Production pipelines	ACID transactions, schema evolution, time travel, upserts	Lakehouse architecture, production tables

📌 **Rule of thumb:** Default to **Parquet** for read-heavy analytical workloads and **Delta** whenever you need reliability, upserts, or auditability in production.

Data Management Principles

Partitioning

Organise data into directory-level partitions (e.g. by `year/month/day`) so that queries only scan the relevant subset of files. Poorly partitioned tables are one of the most common causes of slow Spark jobs and high cloud storage costs.

Schema Enforcement

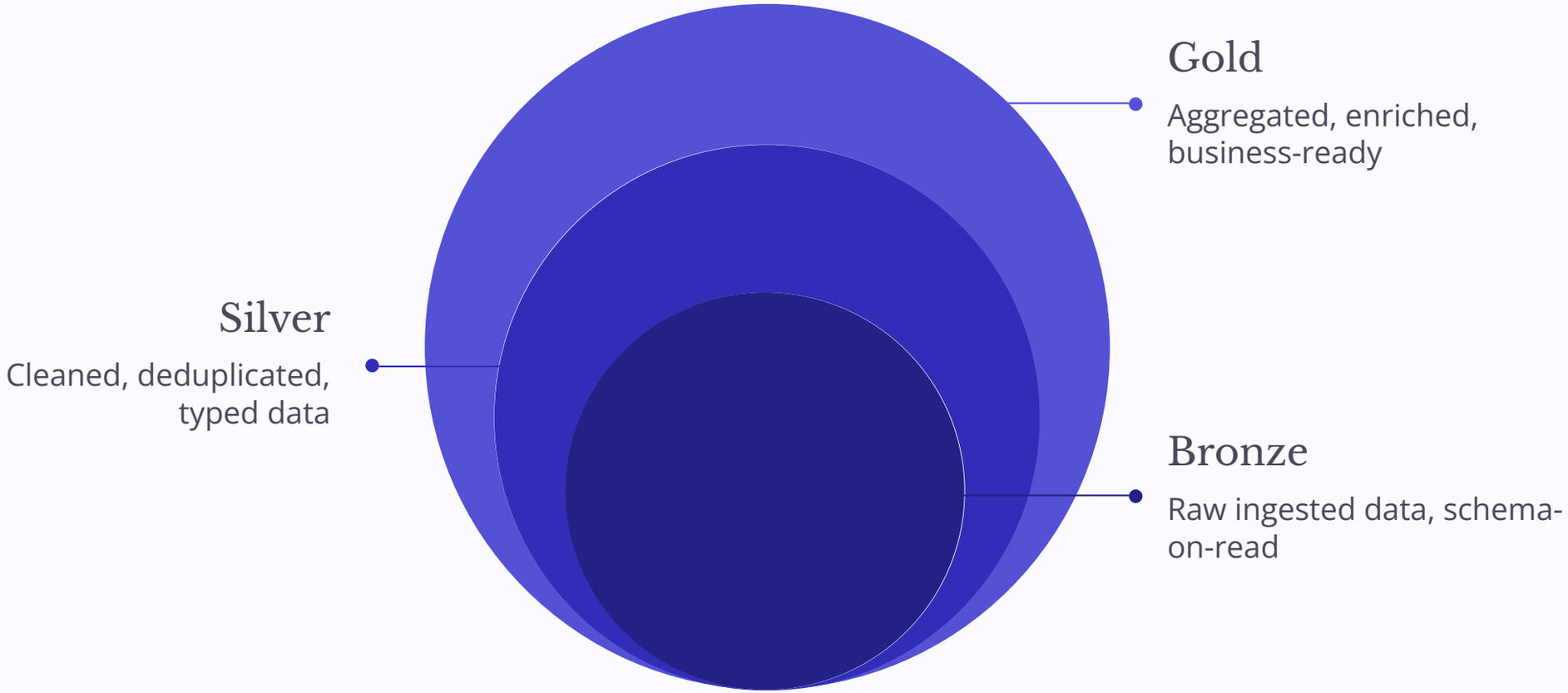
Reject records that do not conform to the expected schema at write time rather than at query time. Delta Lake enforces schema by default, preventing silent data corruption from upstream changes in source systems.

Versioning

Delta Lake's transaction log maintains a full history of all writes, enabling time travel queries and rollback to any prior state. This is essential for debugging pipelines, auditing data changes, and recovering from accidental overwrites.

Storage Patterns — The Medallion Architecture

The Medallion Architecture is a layered approach to organising data in a lakehouse. Each layer applies progressively stricter quality standards, separating concerns and making pipelines easier to debug and maintain.



Raw data lands in **Bronze** exactly as received. **Silver** applies cleansing, deduplication, and type casting. **Gold** contains business-level aggregations and curated datasets ready for BI tools, ML models, and dashboards. This separation ensures that any layer can be reprocessed independently without corrupting others.

What Comes Next

> Most of the pipeline is built — now it's about surfacing insights to business users.

01

✅ Data Sources & Management

ingestion patterns, file formats, DataFrameReader/Writer.

02

✅ Medallion Architecture

Bronze/Silver/Gold layering, data quality, lakehouse storage.

03

✅ Streaming Pipelines

Structured Streaming, Kafka ingestion, stateful aggregations, watermarking.

04

✅ SQL & Data Warehousing

SQL warehouse, query engines, Delta Lake tables.

05

➡️ Business Consumption Layer

How analysts and business users consume Gold-layer data through BI dashboards, data visualizations, reporting tools, and semantic layers.